

GENAI FOR TEST MANAGEMENT • TRACK 2

Module 2

Live Jira Data & Automated Reporting — MCP + n8n

Continues from Module 1's Sprint 14 thread — the same AEB false-positive defect (QAM-207) is now QAM-315, still open in Sprint 15's live sandbox data.

Slides 1–9 • Hands-on lab included • Part A ~40 min • Part B ~50 min

From Static Files to Live Data: Why MCP

Module 1 used files you handed to Copilot. Real Jira data changes hourly.

Static Files

Sprint/backlog exports you attach by hand



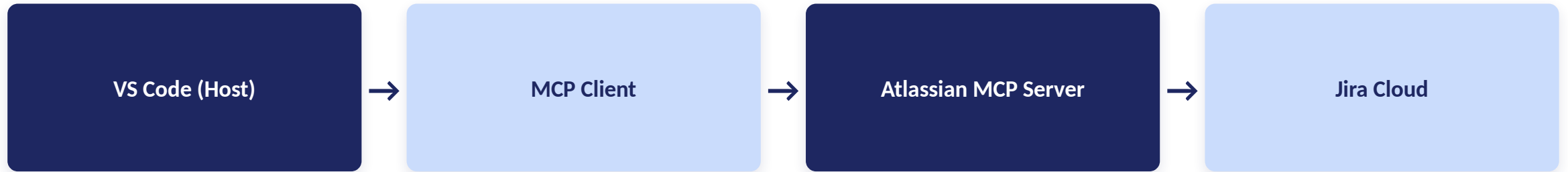
MCP

Live Systems (Jira)

Copilot reaches current sprint/backlog data directly

"MCP is like a USB-C port for AI applications." — modelcontextprotocol.io

How MCP Works (Just Enough Architecture)



Tools

Actions it can run — e.g., search issues

Resources

Data it can hand back — e.g., issue records

Prompts

Ready-made templates — e.g., a sprint-summary prompt

Connecting Copilot to Jira: The Atlassian Remote MCP Server

1

Products connected

Jira, Confluence, JSM, Bitbucket, Compass — official Atlassian-maintained server.

2

Authentication

OAuth 2.1 (sign in, respects existing permissions) or API tokens (admin-provisioned).

3

Scope

Everything visible is scoped to your own Jira permissions — no back-door access.

i

In regulated / automotive environments

Jira is often the agile work-tracking layer sitting on top of a requirements system of record (e.g., DOORS, Polarion) that holds ASIL-tagged requirements. The same MCP connection pattern applies to whichever system is your actual source of truth — confirm with your tooling/compliance team before connecting a lab exercise's output to anything beyond Jira.

Querying Jira Through Copilot: Copilot-Assisted JQL

Describe the ask in plain English; Copilot drafts the JQL; MCP runs it live

YOU ASK

Write a JQL query for issues in QAM that are type = Bug, status is not Done, and priority is Critical or High. Explain the query, then run it.



COPILOT DRAFTS JQL

```
project = QAM AND type = Bug  
AND status != Done  
AND priority in (Critical, High)
```

YOU ASK

Now write JQL for everything in the current sprint with status = Blocked.



COPILOT DRAFTS JQL

```
project = QAM  
AND sprint in openSprints()  
AND status = Blocked
```

Synthesizing a Coverage-Velocity / Defect-Density View

- One MCP query → ask Copilot to combine sprint, backlog, and defect data into a narrative
- Velocity + defect density, computed together from the same pulled data
- Still advisory — the Test Manager confirms before quoting upward

Sprint Velocity

24%

10 of 42 pts (Sprint 15)

Defect Density

6.0

open defects / 10 pts delivered

Backlog Health

50%

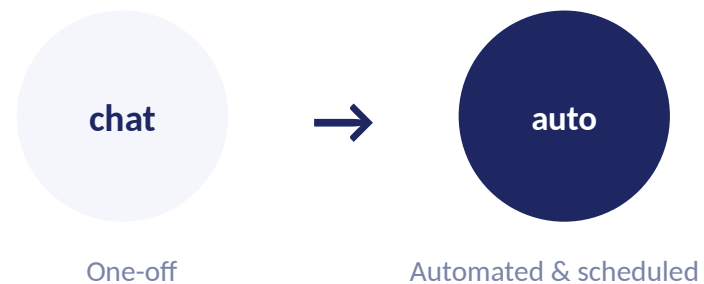
4 of 8 backlog items Ready

Flagged risk: defect density is high relative to velocity — 2 of 6 Sprint 15 defects are Critical and Blocked (QAM-305, QAM-306).

Computed from the seeded Jira sandbox data (sprint-issues.csv, defects.csv, backlog.csv) — sample training data, not a live production metric.

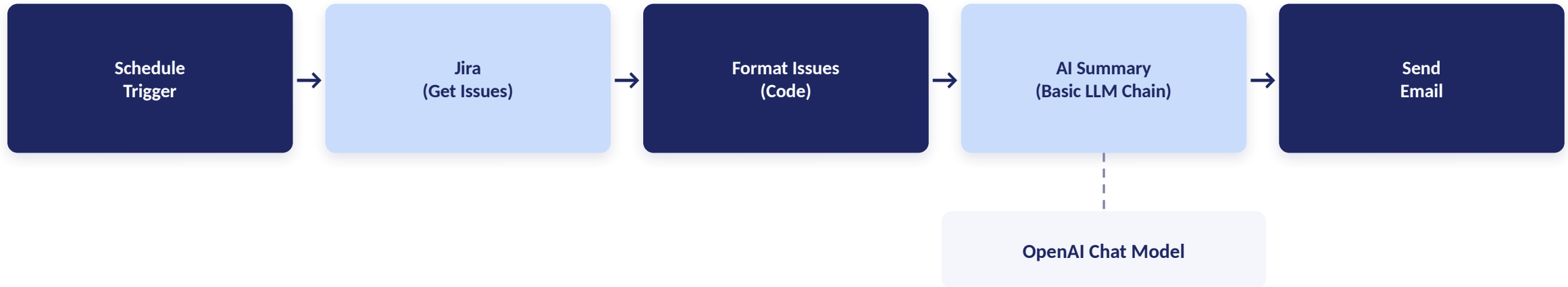
From One-Off Chat to a Standing Workflow

- A great chat answer today doesn't help next Monday — this is the gap n8n closes
- n8n: scheduled trigger → pull data → AI step → deliver output, unattended
- Today: extend a provided starter workflow, don't build from scratch



Inside the n8n Workflow

Schedule Trigger → Jira → Format (Code) → AI Summary → Send Email — verified against a live n8n 2.35.3 container



MODEL INPUT — separate connection type, not the data flow

1. Runs every 15 min for the lab (production cadence would be daily/weekly)
2. Jira node pulls Sprint 15 issues via JQL — same pattern as Slide 5
3. Code node compacts the raw issue array so the AI prompt stays small
4. AI Summary produces the Slide 6 narrative; Send Email delivers it over SMTP

Module 2 Lab



Part A: Query live data via MCP + Copilot-assisted JQL; synthesize the summary



Part B: Wire the same logic into the provided n8n starter workflow



Outcome: a working scheduled report landing in your inbox

Lab data ready

OK

sprint-issues.csv · backlog.csv · defects.csv seeded into the shared QAM Jira sandbox pre-session · n8n-workflow-starter.json — Docker + import verified against a live n8n container. Participants supply their own Jira/AI-provider/SMTP credentials per 00-setup/setup-guide.md.